https://github.com/whiteys1/SoftwareEngineering

[ Git Analyze ]

# 프로젝트 관리 계획서

## - 신영섭 -

문서번호 : [신영섭]프로젝트관리계획서_260412_Doc-002

소　　속 : 소프트웨어학과

팀　　명 : Git Analyze

팀　　원 :

교　　수 : 최차봉 교수님

# 제/개정 이력

| 버전 | 날짜 | 작성자 성명 | 제/개정사항 | 비 고 |
|---|---|---|---|---|
| 1.0 | 2026-04-12 | 신영섭 | 프로젝트 관리 계획서 최초 작성 | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |

# 목 차

1. 서론 ---------- 1
1.1 문서의 목적 및 범위 ---------- 1
1.2 프로젝트 개요 ---------- 1
1.3 프로젝트 목표 ---------- 2
1.4 용어 정의 ---------- 2
1.5 참조 문서 ---------- 2

2. 개발 계획 ---------- 3
2.1 개발 절차 모형 ---------- 3
2.2 WBS(Work Breakdown Structure) ---------- 3
2.3 개발 내용 ---------- 4
2.4 개발 일정 ---------- 5

3. 팀 구성 ---------- 6
3.1 팀 구조 ---------- 6
3.2 역할 및 책임 ---------- 6

4. 품질 관리 ---------- 7
4.1 팀 미팅 계획 ---------- 7
4.2 변경 사항 관리 방법 ---------- 7
4.3 산출물 관리 방법 ---------- 8

5. 개발 환경 ---------- 9
5.1 하드웨어 환경 ---------- 9
5.2 소프트웨어 환경 ---------- 9
5.3 기타 개발 환경 ---------- 9

6. 산출물 ---------- 10
6.1 산출물 정의 ---------- 10
6.2 산출물 작성일 및 담당자 ---------- 10
6.3 기타 제약사항 ---------- 11

7. 기타 사항 ---------- 11
7.1 리스크 관리 ---------- 11
8. 참고문헌 및 부록 ---------- 11

# 1. 서 론

## 1.1 문서 목적 및 범위

본 프로젝트 계획서는 Git Analyze 시스템의 개발에 대한 전반적인 내용을 정리하여 프로젝트를 진행하는 데 있어 각 세부 사항의 지표로 삼는 것을 목적으로 한다.

본 문서는 다음과 같은 범위의 지표로 적용할 수 있다.

① 프로젝트 개요 (프로젝트 정의, 주요 기능 정리)
② 개발 계획 및 환경 (개발 절차 모형, 내용 및 일정, 소프트웨어/하드웨어 환경)
③ 팀 구성 (팀 구조/역할 및 책임)
④ 품질관리 (팀 미팅, 변경 사항 및 산출물 관리)
⑤ 산출물 (산출물 정의, 작성일 및 담당자)

## 1.2 프로젝트 개요

### 1.2.1 프로젝트 정의

Git Analyze는 웹 기반 서비스로, 사용자가 GitHub 레포지토리 URL을 입력하면 GitHub API를 통해 코드와 구조 데이터를 수집하고, LLM을 활용하여 분석·요약 결과를 제공한다.

### 1.2.2 주요 기능 설명

| ID | 설명 |
|---|---|
| F01 | GitHub 레포지토리URL을 입력하면 디렉토리 구조, 파일별 역할, 기술 스택, 의존성을 자동으로 분석한다. |
| F02 | 처음 접하는 레포지토리의 목적, 아키텍처, 핵심 모듈 간 관계를 요약하여 제공한다. |
| F03 | 자연어 질문을 통해 특정 기능이 구현된 파일과 코드 위치를 확인할 수 있다. RAG 기반 코드 임베딩 검색을 활용한다. |
| F04 | GitHub OAuth를 통해 로그인하고, 공개 및 비공개 레포지토리 모두 접근할 수 있다. |
| F05 | 본인의 레포지토리인 경우 문서화 보완 및 구조 개선을 제안한다. |

## 1.3 프로젝트 목표

Git Analyze는 GitHub 레포지토리의 구조와 코드를 자동으로 분석·정리하여, 사용자가 레포지토리의 전체 구조를 빠르게 파악하고 원하는 코드를 즉시 찾을 수 있도록 지원하는 서비스이다. 본인의 레포지토리 관리와 타인의 레포지토리 이해라는 두 가지 사용 시나리오를 모두 지원하는 것을 목표로 한다.

## 1.4 용어 정의

본 문서의 이해를 돕기 위해 사용된 모든 용어 및 약어를 설명하고 정의합니다.

| 용어 | 설명 |
|---|---|
| LLM | Large Language Model. 대규모 언어 모델로, 자연어 이해 및 생성에 사용되는 AI 모델 |
| RAG | Retrieval-Augmented Generation. 검색증강 생성 기법으로, 외부 데이터를 검색하여 LLM 응답 품질을 향상시키는 방식 |
| Github API | GitHub에서 제공하는REST/GraphQL API로, 레포지토리 데이터에 프로그래밍 방식으로 접근 가능 |
| OAuth | Open Authorization. 사용자 인증 및 권한 부여를 위한 개방형 표준프로토콜 |
| 임베딩 | 텍스트 등의 데이터를 고차원 벡터로 변환하는 기법으로, 유사도 검색에 활용 |

## 1.5 참조 문서

[신영섭]시스템정의서_260320_Doc-001.hwp

# 2. 개발 계획

## 2.1 개발 절차 모형

1) 소프트웨어를 개발하기 위한 개발 절차 모형

요구사항이 초기에 명확히 정의되어있으므로, 반복적 개선이 가능한 Iterative(반복적) 개발 모델을 채택한다. 각 반복(Iteration)마다 핵심 기능을 우선 구현한 뒤, 점진적으로 기능을 확장하고 품질을 개선하는 방식으로 진행한다.

2) 개발 절차 모형과 함께 분석, 설계를 진행하기 위한 방법론

프론트엔드와 백엔드가 분리된웹 서비스 구조이며, GitHub API 연동, LLM 호출, RAG 파이프라인 등 여러 외부 서비스와의 통합이 필요하다. 따라서 컴포넌트 기반 설계를 적용하여 각 모듈의 독립성을 확보하고, API 인터페이스 중심으로 설계를 진행한다.

## 2.2 WBS(Work Breakdown Structure)

## 2.3 개발 활동

| 개발 활동 | 설명 |
|---|---|
| 프로젝트 요약 | 프로젝트 요약서 작성 |
| 프로젝트 계획 | 프로젝트 관리 계획서 작성 |
| 요구사항 정의 | 기능/비기능 요구사항 정의서 작성 |
| 요구사항 분석 | 요구사항 분석서 작성 및 우선순위 결정 |
| 아키텍쳐 설계 | 시스템 전체 구조 및 컴포넌트 간 인터페이스 설계 |
| Iteration 1 | GitHub OAuth 인증 및 레포지토리 목록 조회 기능 구현 |
| Iteration 2 | 레포지토리 구조 분석 및 LLM 기반 요약 기능 구현 |
| Iteration 3 | RAG 기반 코드 Q&A기능 구현 |

| | |
|---|---|
| Iteration 4 | 레포지토리 문서화 제안 기능 및 UI 개선 |
| 테스트 | 단위/통합/사용자 테스트 수행 및 버그 수정 |
| 배포 및 종료 | 클라우드 배포, 최종 문서화, 발표 자료 작성 |

## 2.4 개발 일정

| 활동 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 |
|---|---|---|---|---|---|---|---|---|
| 프로젝트 계획 | | | | | | | | |
| 요구사항 정의 및 분석 | | | | | | | | |
| 아키텍처 설계 | | | | | | | | |
| Iteration 1 | | | | | | | | |
| Iteration 2 | | | | | | | | |
| Iteration 3 | | | | | | | | |
| Iteration 4 | | | | | | | | |
| 테스트 | | | | | | | | |
| 배포 및 종료 | | | | | | | | |

# 3. 팀 구성

## 3.1 팀 구조

팀장의 역할은 있지만 모든 멤버가 상호 협력하고 서로의 개발 활동을 확인하며 진행하는 수평적 팀 구조를 채택하였다.

## 3.2 역할 및 책임

| 이름 | 역할 | 구분 | 책임 |
|---|---|---|---|
| 신영섭 | PM | 공동 | 프로젝트 전반 일정 관리, 리스크 관리, 의사결정 |
| | 백엔드 개발 | 개별 | GitHub API 연동, LLM호출, RAG 파이프라인, REST API 구현 |
| | 프론트엔드 개발 | 개별 | React 기반 웹 UI 설계 및 구현 |
| | 인프라/배포 | 개별 | AWS/GCP 클라우드 환경 구성, CI/CD 파이프라인 구축 |
| | 테스트 | 개별 | 단위/통합/사용자 테스트 계획 수립 및 수행 |
| | 문서관리 | 공동 | 산출물 작성, 버전 관리,GitHub Repository 관리 |

# 4. 품질 관리

## 4.1 팀 미팅 계획

(1) Sprint Planning Meeting
- 팀 미팅 시간 : 매주 월요일 오전
- 토의 내용 : 해당 주차 목표 설정, 작업 항목 선정 및 우선순위 결정
- 미팅 참석자 : PM, 개발

(2) Daily Scrum Meeting
- 팀 미팅 시간 : 매일 오전, 10분 내외 진행
- 토의 내용 : 어제 한 일, 오늘 할 일, 장애 현상 등을 공유
- 기록 방법 : GitHub Issues 또는 프로젝트 보드에 기록
- 미팅 참석자 : PM, 개발

(3) Sprint Review
- 팀 미팅 시간 : 각 Iteration 종료 시점
- 토의 내용 : 구현 결과물 리뷰, 기능 동작 확인, 목표 달성 여부 점검
- 미팅 참석자 : PM, 개발

(4) Sprint Retrospective
- 팀 미팅 시간 : 각 Iteration 종료 시점 (Sprint Review 이후)
- 토의 내용 : 개선할 것, 잘한 것, 새로 시작할 것 도출
- 기록 방법 : 회고 내용을 GitHub Wiki 또는 문서로 기록

(5) 비정기 회의
- 프로젝트 진행에 문제가 발생하여 회의가 필요한 경우에 진행된다.
- 비정기 회의는 팀장 또는 지도교수 또는 멘토에 의해 요청될 수 있다.
- 미팅 참석자 : 팀원, 팀장, (지도교수, 멘토)

## 4.2 변경사항 관리 방법

- 변경 사항 발생 시 GitHub Issues에 변경 사유, 영향 범위, 변경 전후 내용을 기록한다.
- 변경 사항은 별도 브랜치에서 작업 후 Pull Request를 통해 main 브랜치에 병합한다.
- 주요 변경 사항(기능 추가/삭제, 아키텍처 변경)은 관련 산출물에 반영하고 제/개정 이력을 갱신한다.

- 변경 이력은 Git 커밋 로그와 문서 내 제/개정 이력으로 이중 관리한다.

## 4.3 산출물 관리 방법

(1) 산출물에 대한 명명(Naming) 방법

- 소프트웨어 : 버전_해당 버전의 산출 일자
  (ex. v1_2026_04_12.exe)
- 문서 : [팀명]문서명_YYMMDD_Doc-해당 문서의 버전
  (ex. [신영섭]프로젝트관리계획서_260412_Doc-001.hwp)
- 그 외 파일 : 해달 파일의 목적이 명확하게 들어나는 이름
  (ex. 260412_video.h264)

(2) 산출물의 버전 관리 방법 및 버전관리 담당자

- 소프트웨어 : GitHub를 통한 Git 버전 관리, main/develop/feature 브랜치 전략 적용 (담당자: 팀원 전체)
- 문서 : 파일명 맨 뒤에 날짜를 기재하고, 문서 내에 제/개정 이력을 수정한다. (담당자 : 신영섭)
- 그 외 파일 : 파일명 앞에 날짜가 붙으며 필요할 경우 1,2 등의 숫자를 추가하여 기재한다.
  (담당자 : 전체)

(3) 산출물의 저장 방법

- 소프트웨어 : Github
- 문서 : 하드웨어 및 팀원 공유 클라우드서버
- 그 외 파일 : 하드웨어 및 팀원 공유 클라우드

# 5. 개발 환경

## 5.1 하드웨어 개발 환경

| 구분 | spec |
|---|---|
| 개발 PC | Apple MacBook Pro(M3pro), RAM18GB, SSD 512GB |
| 클라우드 서버 | AWS(Lmabda, API Gatewat, S3, ECS 등) / GCP (필요시) |

## 5.2 소프트웨어 개발 환경

| 구분 | type | spec |
|---|---|---|
| OS | mac OS | macOS Sequoia |
| 프론트엔드 | Framework | React + TypeScript |
| 백엔드 | Framework | FastAPI(Python) 또는 Sprign Boot |
| LLM | API | Open API (GPT-4o) |
| 벡터 DB | Database | ChromaDB |
| 인증 | Protocol | Github OAuth 2.0 |
| 버전 관리 | Tool | Git +Github |
| CI/CD | Tool | Github Actions |
| 클라우드 | Platform | AWS(ECS, Lmabda, S3, API Gateway) |
| IDE | Tool | IntelliJ IDEA / VS Code |

## 5.3 기타 개발 환경

- Docker : 로컬 개발 환경 컨테이너 화
- Terraform : 클라우드 인프라 IaC 관리
- Postman : API 테스트

# 6. 산출물

## 6.1 산출물 정의

형식 : [팀명]문서명_YYMMDD_Doc-해당 문서의 버전

ex)
- [신영섭]시스템정의서_YYMMDD_Doc-해당 문서의 버전
- [신영섭]프로젝트관리계획서_YYMMDD_Doc-해당 문서의 버전
- [신영섭]요구사항정의서_YYMMDD_Doc-해당 문서의 버전
- [신영섭]요구사항분석서_YYMMDD_Doc-해당 문서의 버전
- [신영섭]소프트웨어설계서_YYMMDD_Doc-해당 문서의 버전

## 6.2 산출물 작성일 및 담당자

| 산출물 | 최종 작성일 | 최종 검토자 | 작성 책임자 |
|---|---|---|---|
| [신영섭]시스템정의서 | 2026-03-20 | | |
| [신영섭]프로젝트관리계획서 | 2026-04-12 | | |

## 6.3 기타 제약사항

- LLM API 호출 비용이 발생하므로 개발 단계에서는 API 호출 횟수를 제한하여 비용을 관리한다.
- GitHub API Rate Limit(인증 시 5,000 req/hr)을 고려한 설계가 필요하다.

# 7. 기타 사항

## 7.1 리스크 관리

| No | Risk | Critical | Occur | Mitigation Plan | Alternative | Respon |
|---|---|---|---|---|---|---|
| 1 | LLM API 비용 초과 | High | Medium | API 호출 캐싱, 토큰 수 제한, 저비용 모델 병행 사용 | 오픈소스 LLM(Ollama) 로컬 배포로 전환 | 신영섭 |
| 2 | GitHub API Rate Limit | Medium | Medium | 응답 캐싱, 요청 최적화, 인증 토큰 사용 | GraphQL API로 전환하여 요청 횟수 절감 | 신영섭 |
| 3 | 일정 지연 | High | Medium | 주간 점검을 통한 초기 식별, 우선순위 기반 기능 축소 | 핵심 기능(구조 분석)만 우선 완성 후 점진적 확장 | 신영섭 |
| 4 | 벡터 DB 성능 부족 | Medium | Low | 인덱싱 최적화, 청크 크기 조정 | 검색 방식을 키워드 기반으로 전환 | 신영섭 |
| 5 | 보안 취약점 | High | Low | OAuth 토큰 관리 철저, 환경 변수 분리, HTTPS 적용 | 접근 권한을 공개 레포지토리로 제한 | 신영섭 |

# 8. 참고문헌 및 부록

-부록 1 : 시스템정의서 ([신영섭]시스템정의서_260401_Doc-001)

-유사 서비스 참고 :DeepWiki, repomix, Github GG, ExplainGithub